Atividade: mochila binária

Seleção de equipamentos com algoritmo genético

1. Problema

Uma equipe precisa escolher equipamentos para transportar em uma mochila. Cada item possui um peso e um valor de utilidade. A mochila tem capacidade limitada: escolha uma combinação que maximize a utilidade total sem ultrapassar essa capacidade.

Cada item pode ser escolhido **uma única vez**. Não é permitido dividir itens. Implemente um algoritmo genético em Python e execute-o nas cinco entradas fornecidas.

Para n itens, represente a decisão por $x_i \in \{0, 1\}$:

$$\text{maximizar } V(x) = \sum_{i=1}^{n} v_i x_i \qquad \text{sujeito a} \qquad P(x) = \sum_{i=1}^{n} p_i x_i \leq C.$$

p_i é o peso, v_i é a utilidade e C é a capacidade. Os pesos estão em kg inteiros; a utilidade é uma pontuação sem unidade.

2. Entradas

Cada arquivo em `entradas/` contém as colunas `item,peso,valor,capacidade`. A capacidade aparece repetida em todas as linhas para que cada CSV seja independente. Leia esse valor uma vez e confira se ele é igual nas demais linhas. Use vírgula como separador.

Arquivo	Itens	Capacidade	Característica
`entrada_01.csv`	10	20 kg	Introdução
`entrada_02.csv`	12	25 kg	Vários itens leves
`entrada_03.csv`	14	30 kg	Itens pesados de alto valor
`entrada_04.csv`	16	35 kg	Pesos e valores próximos
`entrada_05.csv`	20	45 kg	Maior espaço de busca

3. Codificação e avaliação

Use um **vetor binário com um gene por item**, na mesma ordem das linhas do CSV: 1 significa incluir; 0 significa deixar de fora. Um cromossomo como `[1,0,1,0,...]` seleciona o primeiro e o terceiro item.

Calcule peso e valor somando apenas os itens com gene 1. Para simplificar, use esta aptidão:

$$f(x) = \begin{cases} V(x), & P(x) \leq C, \\ 0, & P(x) > C. \end{cases}$$

Uma combinação acima da capacidade não pode ser apresentada como resultado válido. Guarde separadamente a melhor solução **válida** encontrada. Inclua o vetor de zeros na população inicial como uma solução válida de referência.

4. Como organizar o algoritmo

1. **Inicialização:** gere cromossomos aleatórios. Para reduzir o excesso de peso inicial, experimente probabilidade 0,3 de cada gene começar com 1.
2. **Seleção por torneio:** sorteie três indivíduos e escolha aquele com maior aptidão. Repita para escolher o segundo pai.
3. **Cruzamento de um ponto:** sorteie um corte entre dois genes e troque as partes finais dos pais para gerar dois filhos. Sem cruzamento, copie os pais.
4. **Mutação:** para cada gene de cada filho, sorteie se ele será invertido: $0 \leftrightarrow 1$.
5. **Elitismo:** copie o melhor indivíduo da geração sem alteração para a próxima. Preencha as demais posições com os filhos.
6. **Parada:** repita por um número fixo de gerações. Se faltar uma posição na população, adicione somente um dos dois filhos.

Parâmetros iniciais sugeridos: **50 indivíduos, 100 gerações, cruzamento 0,8, mutação 0,05 por gene, torneio de tamanho 3 e um elite**. A taxa de mutação aplica-se a cada gene, não ao cromossomo inteiro.

Divida o código em funções para ler os dados, criar e avaliar a população, selecionar pais, cruzar, mutar e executar o algoritmo. A leitura pode usar `csv.DictReader`; os sorteios podem usar `random`.

5. Experimentos

1. Execute a configuração inicial **cinco vezes em cada entrada**, com sementes 1, 2, 3, 4 e 5.
2. Na entrada 05, repita com taxa de mutação **0,01**, mantendo os demais parâmetros e as mesmas sementes.
3. Ainda na entrada 05, repita com população de **100 indivíduos**, mantendo a mutação inicial de 0,05 e os demais parâmetros.
4. Compare o melhor valor válido de cada execução. Para cada configuração e entrada, apresente o maior valor, a média dos cinco valores e o tempo médio de execução.

Registre, a cada geração, o valor da melhor solução válida encontrada **até aquele momento**. Faça um gráfico dessa evolução para uma execução de cada entrada. Use `time.perf_counter()` para medir a execução do algoritmo, sem incluir a leitura do CSV nem a criação do gráfico.

6. Entrega

- Código Python que permita escolher o CSV, os parâmetros e a semente sem alterar a lógica do algoritmo.
- Tabela dos experimentos e os cinco gráficos de evolução.
- Para a melhor execução de cada entrada: semente, vetor binário, itens selecionados, peso total, valor total e capacidade.
- Respostas curtas: por que execuções diferentes podem produzir resultados diferentes? Como o elitismo afeta a evolução? O que mudou ao reduzir a mutação e ao aumentar a população?

7. Cuidados

Não selecione pais sempre pela posição na lista. Não modifique os pais ao criar os filhos: trabalhe com cópias. Confira se todos os genes são binários, se o tamanho do cromossomo corresponde ao número de itens e se a população mantém o tamanho configurado.

Encontrar uma boa solução não comprova que ela é ótima. Não é necessário implementar busca exaustiva nem conhecer a solução ótima para concluir a atividade.